

实验报告成绩:	成绩评定日期:
---------	---------

2025 ~ 2026 学年秋季学期

《计算机系统》必修课

课程实验报告



班级：人工智能（一）2302

组长：贝冠霖

组员：胡中才

报告日期：2025.12.28

目录

一、 项目介绍	3
二、 流水线各个阶段的说明:	5
三、 组员感受以及改进意见	18
四: 参考文献	19

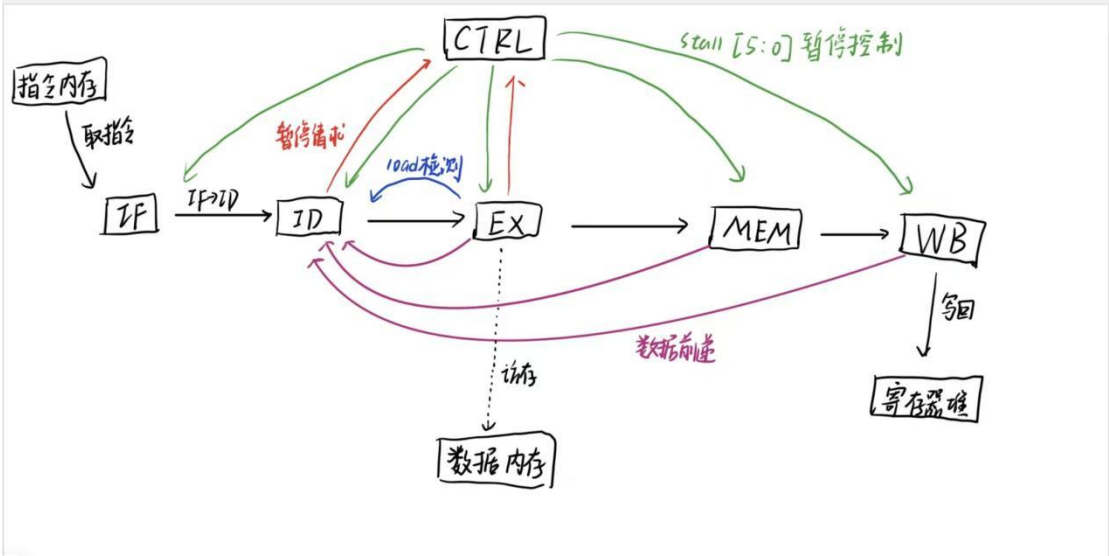
一、项目介绍

1.1 成员工作量以及工作占比

姓名	完成任务点	任务量占比
贝冠霖	ID. v 文件拓展, IF. v 拓展, 写出 mymul 的乘法计数器功能, 实现了数据前递, 指令拓展, 数据相关, 协调各模块接口, 集成 HI/L0 寄存器访问等功能	50%
胡中才	EX. v, CTRL. v, MEM. v, regfile. v 等文件拓展, 实现了乘除法、访存指令、数据对齐, 数据前递输出, 以及实现流水线暂停控制增加了 HI/L0 寄存器等, 报告撰写	50%

1.2 总体设计

不同流水段之间的连线图:



我们设计了一个完整的五级流水线 MIPS 处理器, 核心包含取指 (IF)、译码 (ID)、执行 (EX)、访存 (MEM) 和写回 (WB) 五个阶段, 并辅以流水线控制 (CTRL) 模块和顶层集成模块。该处理器拥有 32 个 32 位通用寄存器和 HI/L0 特殊寄存器, 采用小端字节序和 32 位数据地址总线, 并集成了自主设计的 32

周期移位乘法器以支持多周期乘除运算，能够顺利通过全部 64 个功能测试点验证。

在指令集方面，处理器完整实现了 47 条 MIPS 指令，涵盖算术逻辑运算、移位比较、乘除操作、跳转分支及内存访问等类型。各流水阶段通过专用数据总线顺序连接构成主执行通路，同时为高效处理数据冒险，我们引入了多层次数据前递系统，新增了从 EX、MEM 和 WB 阶段到 ID 阶段的三条前递通路，采用“最近优先”的优先级策略确保最新产生的操作数能被及时使用。

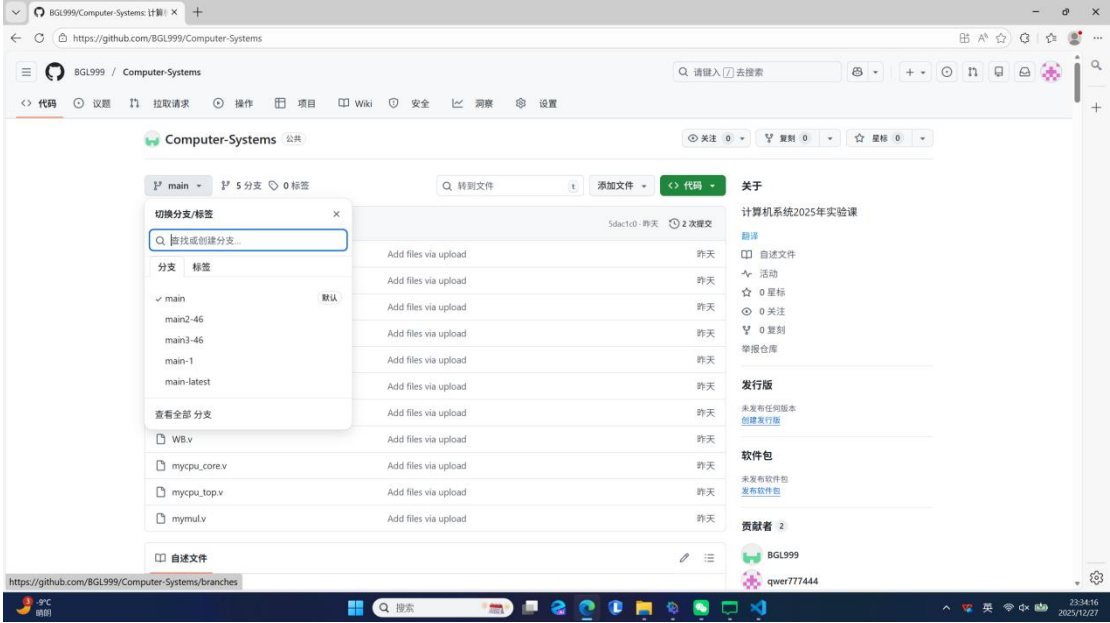
控制流处理上，ID 阶段负责解析指令并生成分支控制信号送往 IF 阶段以更新程序计数器，而 CTRL 模块则统一处理来自 ID 和 EX 阶段的暂停请求，通过精确的流水线调度机制应对 load-use 冒险和乘除法多周期操作。内存系统支持字节、半字、字的对齐访问，自动处理符号扩展和零扩展，符合 MIPS 架构规范。

处理器设计强调模块化与可扩展性，各阶段功能独立且接口清晰，便于调试与后续升级。自主设计的乘法器与流水线紧密集成，提供精确的暂停控制；完整的调试接口支持丰富的状态信号输出，便于性能分析和错误定位。

整体而言，该处理器在保证功能完整性的基础上，通过数据前递和精确暂停控制等优化技术，实现了较高性能指令执行，为体系结构教学与研究提供了良好平台，并为未来功能扩展和性能优化奠定了坚实基础。我们设计了一个完整的五级流水线 MIPS 处理器，核心包含取指（IF）、译码（ID）、执行（EX）、访存（MEM）和写回（WB）五个阶段，并辅以流水线控制（CTRL）模块和顶层集成模块。该处理器拥有 32 个 32 位通用寄存器和 HI/LO 特殊寄存器，采用小端字节序和 32 位数据地址总线，并集成了自主设计的 32 周期移位乘法器以支持多周期乘除运算，能够顺利通过全部 64 个功能测试点验证。

开发环境方面，我们采用 Vivado 2019.2 作为仿真与综合工具，配合 VS Code 代码编辑器和 Git 版本控制系统。

Github 版本控制



二、流水线各个阶段的说明：

2.1 取指阶段（IF）

2.1.1 整体功能说明

取指阶段负责从指令存储器中获取下一条待执行指令，并计算下一条指令的地址。作为流水线的起始阶段，IF 阶段的核心任务是保证指令流的连续供给，同时正确处理分支跳转指令带来的控制流变化。该阶段采用分支延迟槽机制，确保分支指令后的指令总是被执行，从而简化流水线控制复杂度。

2.1.2 关键端口介绍

端口名称	方向	位宽	功能描述
clk	输入	1	系统时钟信号，上升沿触发
rst	输入	1	异步复位信号，高电平有效
stall	输入	6	流水线暂停控制信号，控制各阶段运行状态
br_bus	输入	33	分支控制总线，包含分支使能和目标地址
if_to_id_bus	输出	33	送往 ID 阶段的总线，包含有效标志和当前 PC 值

2.1.4 核心功能模块

1. 程序计数器（PC）管理单元

PC 寄存器在时钟上升沿更新，其值来源由多路选择器控制：

```
assign next_pc = br_e ? br_addr : pc_reg + 32'h4;
```

该选择器根据分支使能信号决定下一条指令地址是分支目标地址还是顺序地址（PC+4）。在流水线暂停时，通过 stall[0]信号控制 PC 保持不变。

2. 指令存储器接口控制器

控制与指令 SRAM 的交互：

```
verilog
assign inst_sram_en = ce_reg;
assign inst_sram_addr = pc_reg;
assign inst_sram_wen = 4'b0; // 只读不写
接口设计简单高效，仅当指令有效（ce_reg 为 1）时才使能存储器，地址直接使用 PC 值，写使能始终为 0 表示只读操作。
```

2.2 译码阶段（ID）

2.2.1 整体功能说明

译码阶段是流水线的控制中心，负责解析 32 位指令字，生成各流水阶段所需的控制信号，读取寄存器操作数，并处理数据前递和分支判断。本阶段采用四级优先级数据前递机制，有效解决数据相关性问题，同时集成分支条件判断逻辑，支持多种条件分支指令。

2.2.2 主要端口介绍

接口名	宽度	输入/输出	作用说明
clk	1	输入	时钟信号
rst	1	输入	复位信号
stall	6	输入	控制暂停信号
ex_is_load	1	输入	判断上一条指令是否是加载指令
if_to_id_bus	33	输入	IF 段发给 ID 段的数据

接口名	宽度	输入/输出	作用说明
inst_sram_rdata	32	输入	当前指令地址中储存的值
wb_to_rf_bus	38	输入	WB 段传给 ID 段要写入寄存器的值
ex_to_id	38	输入	EX 段传给 ID 段的数据，用于判断数据相关
mem_to_id	38	输入	MEM 段传给 ID 段的数据，用于判断数据相关
wb_to_id	38	输入	WB 段传给 ID 段的数据，用于判断数据相关
hilo_ex_to_id	66	输入	EX 段传给 ID 段要写入乘除法寄存器的值
stallreq	1	输出	控制暂停信号
id_to_ex_bus	169	输出	ID 段传给 EX 段的数据
br_bus	33	输出	ID 段传给 IF 段的数据，用于确定下一条指令地址
stallreq_from_id	1	输出	从 ID 段发出的暂停请求信号

2.2.3 核心功能模块

1. 指令译码器

采用两级译码结构：操作码译码和功能码译码。首先根据指令高 6 位（opcode）进行初步分类，对于 R 型指令进一步根据低 6 位（funct）确定具体操作。译码器输出 47 个指令类型信号，每个信号对应一条具体指令。

2. 数据前递选择器

实现四级优先级的数据选择逻辑：

//三级数据前递选择器

```
assign rdata11 = (inst_mfhi | inst_mflo) ? mf_data
                : (ex_id_we &(ex_id_waddr==rs)) ? ex_id_wdata
                : (mem_id_we &(mem_id_waddr==rs)) ? mem_id_wdata
                : (wb_id_we &(wb_id_waddr==rs)) ? wb_id_wdata
                : rdata1;
assign rdata22 = (inst_mfhi | inst_mflo) ? mf_data
                : (ex_id_we &(ex_id_waddr==rt)) ? ex_id_wdata
                : (mem_id_we &(mem_id_waddr==rt)) ? mem_id_wdata
                : (wb_id_we &(wb_id_waddr==rt)) ? wb_id_wdata
                : rdata2;
```

该选择器遵循“最近写优先”原则，确保使用最新产生的数据，最大程度减少流水线暂停。

3. 分支判断单元

支持 8 种分支条件判断：

```
assign pc_plus_4 = id_pc + 32'h4;
assign rs_ge_z    = (rdata11[31] == 1'b0);
assign rs_gt_z    = (rdata11[31] == 1'b0 & rdata11 != 32'b0 );
assign rs_le_z    = (rdata11[31] == 1'b1 | rdata11 == 32'b0 );
assign rs_lt_z    = (rdata11[31] == 1'b1);
assign rs_eq_rt   = (rdata11 == rdata22); //判断rs和rt是否相等
```

每种分支指令选择相应的条件信号，组合生成分支使能信号 br_e。

4. Load-Use 冒险检测器

检测 load 指令后的数据相关：

// ID阶段产生的暂停请求信号

```
assign stallreq_from_id = (ex_is_load & ex_id_waddr == rs) | (ex_is_load & ex_id_waddr == rt) ;
```

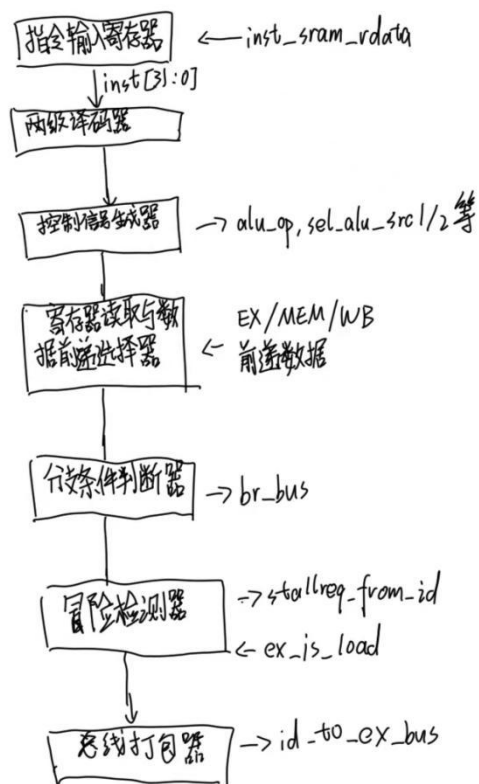
当 EX 阶段为 load 指令且其目标寄存器与 ID 阶段需要的源寄存器重合时，请求暂停流水线。

ID 阶段的数据前递系统采用四级优先级设计，其核心思想是“时空最近优先”。EX 阶段的结果在时间上最新，优先级最高；MEM 和 WB 阶段依次递减；寄存器堆原始值作为最后保障。这种策略能够覆盖绝大多数数据相关场景，显著减少流水线暂停。特别地，对于 HI/LO 寄存器的访问，我们设计了专门的旁路通路，确保乘除法结果能够及时被后续的 mfhi/mflo 指令使用，避免了额外的等待周期。

分支判断逻辑的早期化是本阶段的重要优化。通过在 ID 阶段即完成分支条件计算和目标地址生成，我们将分支指令的代价降低到最小——仅产生一个时钟周期的控制冒险。同时，load-use 冒险检测器精确识别数据相关场景，仅在实

际发生冒险时才请求暂停，这种按需暂停策略避免了不必要的性能损失，提高了流水线整体效率

ID 结构图：



2.3 执行阶段（EX）

2.3.1 整体功能说明

执行阶段是流水线的计算核心，负责所有算术逻辑运算、内存地址计算以及乘除法操作。本阶段集成了 ALU 运算单元、32 周期移位乘法器、多周期除法器，并负责生成内存访问控制信号。通过精确的多周期操作控制，实现在不增加流水线深度的前提下支持复杂运算。

2.3.2 主要端口介绍

接口名	宽度	输入/ 输出	作用说明
clk	1	输入	时钟信号
rst	1	输入	复位信号
stall	6	输入	控制暂停信号

接口名	宽度	输入/输出	作用说明
id_to_ex_bus	169	输入	ID 段传给 EX 段的数据
ex_to_mem_bus	80	输出	EX 段传给 MEM 段的数据
data_sram_en	1	输出	内存数据的读写使能信号
data_sram_wen	4	输出	内存数据的写使能信号
data_sram_addr	32	输出	内存数据存放的地址
ex_to_id	38	输出	EX 段传给 ID 段的数据
data_sram_wdata	32	输出	要写入内存的数据
stallreq_from_ex	1	输出	EX 发出的是否暂停的信号
ex_is_load	1	输出	EX 段发给 ID 段的数据，用来判断上一条指令是否是储存指令
hilo_ex_to_id	66	输出	EX 段将乘除法器的结果发给 ID 段的 regfile 模块

2.3.3 核心功能模块

1. ALU 运算单元

支持 12 种运算操作：

```
assign {op_add, op_sub, op_slt, op_sltu,
        op_and, op_nor, op_or, op_xor,
        op_sll, op_srl, op_sra, op_lui} = alu_control;
```

操作数选择灵活，支持寄存器值、立即数（符号/零扩展）、PC 值和移位数等多种来源。

2. 内存对齐控制器

处理不同粒度的内存访问：

```

// 数据存储器写使能生成逻辑：根据指令类型和地址对齐确定哪些字节需要写入
assign data_sram_wen =
    // 情况1: 字节存储/加载 (sb/lb/lbu) 指令，根据地址低2位选择字节
    /*4'b0101: 字节加载 (lb/lbu) 4'b1101: 字节存储 (sb) 4'b0111: 半字加载 (lh/lhu)
    (data_ram_reader==4'b0101 && ex_result[1:0] == 2'b00) ? 4'b0001 :
    (data_ram_reader==4'b0101 && ex_result[1:0] == 2'b01) ? 4'b0010 :
    (data_ram_reader==4'b0101 && ex_result[1:0] == 2'b10) ? 4'b0100 :
    (data_ram_reader==4'b0101 && ex_result[1:0] == 2'b11) ? 4'b1000 :

    // 情况2: 半字存储/加载 (sh/lh/lhu) 指令，根据地址低2位选择半字
    (data_ram_reader==4'b0111 && ex_result[1:0] == 2'b00) ? 4'b0011 : // 地址对齐到
    (data_ram_reader==4'b0111 && ex_result[1:0] == 2'b10) ? 4'b1100 : // 地址对齐到

    // 情况3: 字存储/加载 (sw/lw) 指令或默认情况
    data_ram_wen; // 使用原始的写使能信号 (通常为4'b1111)

// 数据存储器地址：直接使用ALU计算的结果作为内存地址
assign data_sram_addr = ex_result;

// 数据存储器写入数据对齐处理：根据写使能信号将数据放置到正确的字节位置
assign data_sram_wdata =
    // 情况1: 字存储 (sw)，写入全部4个字节
    data_sram_wen==4'b1111 ? rf_rdata2 : // 直接写入32位数据

    // 情况2: 字节存储 (sb)，根据写使能位置将数据放在正确的字节
    data_sram_wen==4'b0001 ? {24'b0, rf_rdata2[7:0]} : // 存入第0字节 (最低)
    data_sram_wen==4'b0010 ? {16'b0, rf_rdata2[7:0], 8'b0} :
    data_sram_wen==4'b0100 ? {8'b0, rf_rdata2[7:0], 16'b0} :
    data_sram_wen==4'b1000 ? {rf_rdata2[7:0], 24'b0} :

    // 情况3: 半字存储 (sh)，根据写使能位置将数据放在正确的半字
    data_sram_wen==4'b0011 ? {16'b0, rf_rdata2[15:0]} : // 存入低半字 (字节
    data_sram_wen==4'b1100 ? {rf_rdata2[15:0], 16'b0} : // 存入高半字 (字节

    // 默认情况：不写入数据
    32'b0;

```

根据指令类型 (sb/sh/sw) 和地址低 2 位生成对应的字节写使能。

3. 乘法器控制器

32 周期移位乘法器的状态机控制：

```

// 乘法器控制状态机（组合逻辑）
always @ (*) begin // 敏感列表为*, 表示任何输入变化都触发
    if (rst) begin // 复位条件
        stallreq_for_mul = `NoStop; // 复位时无暂停请求
        mul_opdata1_o = `ZeroWord; // 操作数1清零
        mul_opdata2_o = `ZeroWord; // 操作数2清零
        mul_start_o = `MulStop; // 停止乘法器
        signed_mul_o = 1'b0; // 清除有符号标志
    end
    else begin // 正常工作情况
        // 默认值: 无暂停请求, 操作数为0, 乘法器停止, 无符号乘法
        stallreq_for_mul = `NoStop;
        mul_opdata1_o = `ZeroWord;
        mul_opdata2_o = `ZeroWord;
        mul_start_o = `MulStop;
        signed_mul_o = 1'b0;

        // 根据乘法指令类型进行控制
        case ({inst_mult, inst_multu}) // 拼接两位信号形成case条件
            2'b10: begin // 有符号乘法 (mult指令)
                if (mul_ready_i == `MulResultNotReady) begin // 乘法器未就绪

```

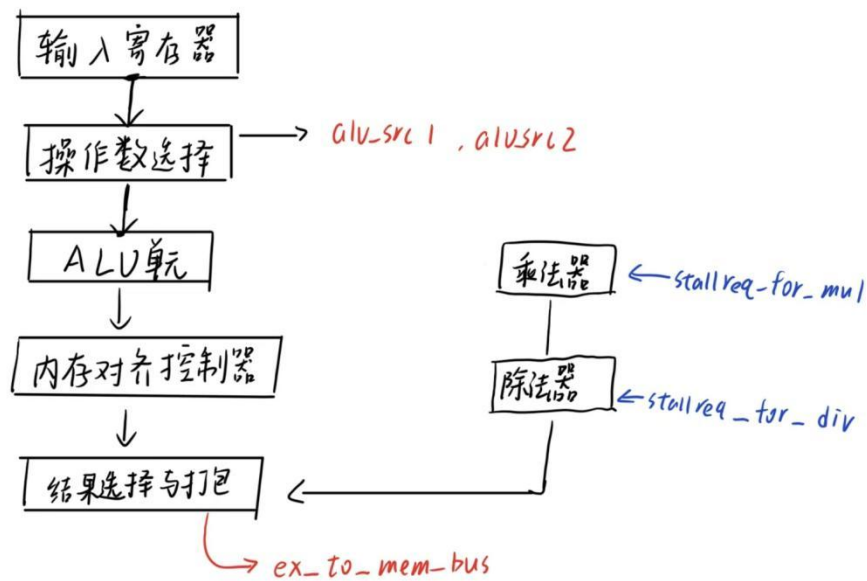
在乘法运算期间请求流水线暂停, 运算完成后恢复。

4. 除法器控制器

类似乘法器的多周期控制, 支持有符号和无符号除法, 结果包含商和余数两部分, 分别写入 LO 和 HI 寄存器。

EX 阶段的乘除法控制器采用了精细化的状态机设计, 实现了计算与暂停的精确协调。乘法器在工作期间会请求流水线暂停, 但除法器还增加了除零异常检测机制。两个多周期单元均支持运算中途取消, 当遇到更高优先级的操作 (如异常处理) 时能够及时响应。这种设计在保证功能正确性的同时, 提供了良好的系统响应性。

内存对齐控制器的设计充分考虑了 MIPS 架构的内存访问特性。对于非对齐的字节/半字访问, 控制器能够自动提取正确的数据片段, 并生成相应的字节使能信号。写入数据的组装逻辑同样精巧, 能够根据目标位置将数据放置到 32 位字的正确字节上。这种完整的内存访问支持使得处理器能够运行标准的 MIPS 程序, 包括需要精细内存操作的应用。



2.4 访存阶段 (MEM)

2.4.1 功能概述

访存阶段负责处理内存读取数据的提取、对齐和符号扩展。本阶段的主要任务是从数据存储器读取的数据中提取正确的部分，并进行适当的扩展，生成最终的写回数据。

2.4.2 核心数据处理

```

// 00100: 1110 (加载半字, 零扩展)
assign rf_wdata =
    // lw指令: 读取整个字(4字节)
    (data_ram_readen==4'b1111 && data_ram_en==1'b1) ? data_sram_rdata :

    // lb指令(有符号扩展): 根据地址低2位选择字节并进行符号扩展
    (data_ram_readen==4'b0001 && data_ram_en==1'b1 && ex_result[1:0]==2'b00) ?
        {{24{data_sram_rdata[7]}}, data_sram_rdata[7:0]} : // 字节0, 符号扩展,
    (data_ram_readen==4'b0001 && data_ram_en==1'b1 && ex_result[1:0]==2'b01) ?
        {{24{data_sram_rdata[15]}}, data_sram_rdata[15:8]} :
    (data_ram_readen==4'b0001 && data_ram_en==1'b1 && ex_result[1:0]==2'b10) ?
        {{24{data_sram_rdata[23]}}, data_sram_rdata[23:16]} :
    (data_ram_readen==4'b0001 && data_ram_en==1'b1 && ex_result[1:0]==2'b11) ?
        {{24{data_sram_rdata[31]}}, data_sram_rdata[31:24]} :

    // lbu指令(零扩展): 根据地址低2位选择字节并进行零扩展
    (data_ram_readen==4'b0010 && data_ram_en==1'b1 && ex_result[1:0]==2'b00) ?
        {24'b0, data_sram_rdata[7:0]} :
    (data_ram_readen==4'b0010 && data_ram_en==1'b1 && ex_result[1:0]==2'b01) ?
        {24'b0, data_sram_rdata[15:8]} :
    (data_ram_readen==4'b0010 && data_ram_en==1'b1 && ex_result[1:0]==2'b10) ?
        {24'b0, data_sram_rdata[23:16]} :
    (data_ram_readen==4'b0010 && data_ram_en==1'b1 && ex_result[1:0]==2'b11) ?
        {24'b0, data_sram_rdata[31:24]} :

```

数据处理原理:

字节选择: 根据地址低 2 位选择正确的字节/半字

符号扩展: 有符号加载指令进行符号扩展

零扩展: 无符号加载指令进行零扩展

直接传递: 字加载和 ALU 结果直接传递

2.4.3 接口信号

信号名称	方向	位宽	功能
data_sram_rdata	输入	32	从数据存储器读取的数据
mem_to_wb_bus	输出	70	到 WB 阶段的数据总线
mem_to_id	输出	38	到 ID 阶段的前递数据

MEM（访存阶段）是五级流水线中处理数据存储器访问的核心模块，其主要功能是根据 EX 阶段计算出的地址和访存指令类型，从数据存储器中读取数据，并进行必要的对齐与格式转换，为写回阶段提供最终的运算结果。

本设计的 MEM 模块实现了三大核心机制：首先，它完成了数据提取与格式转换，能根据 lw、lh、lb 等指令的类型及地址对齐信息，从 32 位存储器数据中精确提取字节或半字，并执行有符号或无符号扩展；其次，作为数据前递通路的关键

键一环，它将访存结果 `mem_to_id` 实时反馈至译码阶段，有效解决后续指令的数据依赖；最后，它集成了结果仲裁逻辑，能够正确选择写回数据——对于加载指令，写回经处理的内存数据，对于其他指令，则直接传递来自 EX 的 ALU 运算结果，确保了处理器在各种场景下的数据一致性。

2.5 写回阶段（WB）

2.5.1 功能概述

写回阶段负责将运算结果写入寄存器堆，并提供完整的调试信息输出。虽然实际的寄存器写入发生在 ID 阶段（通过前递机制），但 WB 阶段仍负责生成正式的写回信号。

2.5.2 主要功能

结果打包：将 MEM 阶段的结果打包成寄存器写回格式

调试接口：提供 PC、写使能、写地址、写数据等调试信号

数据前递：提供到 ID 阶段的最后一级前递数据

2.5.3 接口设计

```
assign wb_to_rf_bus = {
    rf_we,
    rf_waddr,
    rf_wdata
};

assign wb_to_id = {
    rf_we,
    rf_waddr,
    rf_wdata
};

assign debug_wb_pc = wb_pc; // WB阶段的PC值
assign debug_wb_rf_wen = {4{rf_we}}; // 如果写使能为1，设置4个bit为1
assign debug_wb_rf_wnum = rf_waddr; // 写入的寄存器地址
assign debug_wb_rf_wdata = rf_wdata; // 写入的寄存器数据
```

3.5.4 信号说明

信号类型	信号名称	方向	位宽	功能
写回输出	wb_to_rf_bus	输出	38	寄存器写回数据

信号类型	信号名称	方向	位宽	功能
前递输出	wb_to_id	输出	38	到 ID 阶段的前递数据
调试输出	debug_wb_pc	输出	32	写回阶段的 PC 值
	debug_wb_rf_wen	输出	4	寄存器写使能（扩展）
	debug_wb_rf_wnum	输出	5	寄存器写地址
	debug_wb_rf_wdata	输出	32	寄存器写数据

2.6 流水线控制模块（CTRL）

2.6.1 功能概述

流水线控制模块负责接收各阶段的暂停请求，生成统一的流水线暂停控制信号。本模块是流水线的调度中心，确保在数据冒险和多周期操作时，流水线能够正确暂停。

2.6.2 暂停控制逻辑

```

always @ (*) begin
    //如果ex发起stall, 那么ex和之后的都stall
    //如果id发起stall, 那么mem和之后都stall
    if (rst) begin
        stall <= `StallBus'b0;
    end
    else if(stallreq_from_ex == 1'b1) begin
        stall <= 6'b001111;
    end
    else if(stallreq_from_id == 1'b1) begin
        stall <= 6'b000111;
    end else begin
        stall <= 6'b000000;
    end
end
end

```

endmodule

暂停策略:

EX 暂停（乘除法操作）：暂停 EX、MEM、WB 阶段，IF 和 ID 继续

ID 暂停（load-use 冒险）：暂停 ID、EX、MEM、WB 阶段，IF 继续

优先级：EX 暂停优先级高于 ID 暂停

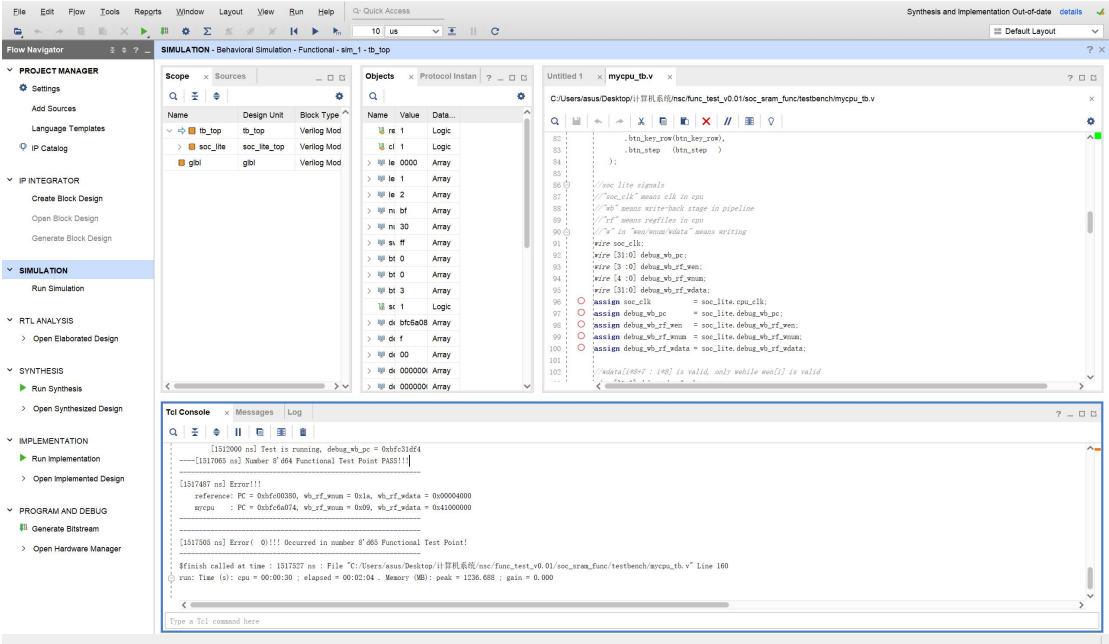
2.6.3 暂停信号定义

信号位	对应阶段	功能说明
stall[0]	PC 保持	PC 寄存器不更新
stall[1]	IF 阶段	IF 阶段暂停
stall[2]	ID 阶段	ID 阶段暂停
stall[3]	EX 阶段	EX 阶段暂停
stall[4]	MEM 阶段	MEM 阶段暂停
stall[5]	WB 阶段	WB 阶段暂停

CTRL（流水线控制）模块是处理器流水线的调度中枢，它不处理具体指令数据，而是负责接收和仲裁来自 ID 阶段（Load-Use 冒险）和 EX 阶段（乘除法多周期操作）的暂停请求，据此生成全局的 6 位 stall 控制信号。这组信号精确地分发至流水线的五个阶段及 PC 寄存器，实现对特定流水段或整个指令流的“冻结”，为冒险的消解和多周期操作的完成提供必要的时间窗口，是保障流水线正确性的关键。

其核心设计在于一套明确的优先级仲裁策略：EX 阶段因乘除法发起的暂停请求优先级高于 ID 阶段因 Load-Use 冒险发起的请求。当请求生效时，它并非简单地暂停全部流水线，而是采取级联暂停策略——例如 EX 暂停时，仅停止 EX 及其后的 MEM、WB 阶段，允许 IF 和 ID 继续取指译码以填充流水线。这种精细化的控制机制最大限度地减少了不必要的流水线“气泡”，在解决冒险的同时优化了处理器的整体吞吐效率。

仿真结果展示：



三、组员感受以及改进意见

贝冠霖感受以及改进意见：

这次 CPU 实验让我学到了很多。我主要负责 ID 模块，这是整个 CPU 里很重要的一部分，要把指令翻译成控制信号，还要处理数据依赖问题。刚开始看到那个只能跑四条指令的模板时，心里有点没底，不知道能不能完成这么多功能。

实际做起来确实不容易。每条 MIPS 指令都要仔细分析它的格式，然后设计对应的控制信号。最麻烦的是数据前递，就是当一条指令的结果马上要被下一条指令使用时，怎么把结果提前传过去。这个逻辑要想清楚优先级，还要和 EX、MEM、WB 模块配合好。

和队友合作很重要。我负责译码部分，队友负责执行和控制部分，我们得经常讨论接口怎么定义，信号怎么传递。比如数据前递的总线格式，就是我们一起商量确定的。调试的时候也互相帮忙，一个人看波形图，另一个人查代码，这样效率高很多。

我觉得实验安排可以再改进一下。课时时间太短，不够完成这项工作；在实验课开始前没有入门引导，对于没接触过硬件的我而言，github 中给的文档内容比较概括，相较于教程，更类似于说明性文档，看完仍然不知道做什么。

胡中才感受以及改进意见：

这次课程设计让我收获巨大，课堂上那些抽象的流水线、冒险冲突的概念，在动手搭建的过程中变得无比清晰和具体。当第一次看到自己写的 CPU 成功跑通测试程序时，那种把理论知识转化为实际可运行系统的成就感，是单纯看书做题无法比拟的。

过程当然不是一帆风顺。刚开始面对一个庞大的工程和全新的硬件描述语言时，我完全不知道从哪里入手，连最基本的仿真和看波形都要从头学起。在调试数据前递和暂停逻辑时，一个小小的优先级错误就会让整个系统行为异常，常常需要花费几个小时在波形图里寻找蛛丝马迹，这个过程非常磨练耐心和细心。

如果未来有机会继续完善，我觉得可以从几个方面改进：一是在项目初期就搭建更友好的仿真测试环境，比如增加一些可视化的调试信息，这样定位问题能更快；二是在架构设计上，或许可以尝试引入更简单的静态分支预测，进一步提升效率；三是团队的代码管理和协作流程可以更规范一些，比如明确模块接口的文档。

回顾整个项目，它带给我的远不止一个能工作的 CPU。更重要的是，我亲身体会到了如何将一个复杂的系统分解成模块、如何团队协作解决问题、以及硬件设计中对精确性的极致要求。这段经历让我对“计算机系统”这个词有了更深的理解，看到了软件指令背后那个精密协作的硬件世界，这对我未来的学习和工作都会是宝贵的财富。

四：参考文献

- 1:雷思磊 著《自己动手写 CPU》 电子工业出版社
- 2:龙芯杯官方的参考文档
- 3:王志英等《计算机体系结构（第二版）》<清华大学出版社>